A. Pie Progress

Input file:	`standard input`
Output file:	`standard output`
Time limit:	15 seconds
Memory limit:	1024 megabytes

Some pies are sweet, full of fruit and jam and sugar.

Some pies are savory, full of meat and potatoes and spices.

Some pies are in fact not pies at all but tarts or galettes. This probably won't stop you from eating them.

Every single day for N days, you're determined to eat a pie for dinner. Every morning, you'll take a trip to your local pie shop, and buy 0 or more of their pies. Every night, you'll eat one pie that you've bought. Pies never go bad, so you don't need to eat a pie on the same day that you bought it. You may instead eat one that you purchased on an earlier day.

On the ith day, the shop has M pies for sale, with the jth of these pies costing $C_{i,j}$ dollars. You can choose to buy any (possibly empty) subset of them. However, this shop has measures in place to protect itself against crazy pie fanatics buying out its products too quickly. In particular, if you buy p pies on a single day, you must pay an additional tax of p^2 dollars.

Input

Input begins with an integer $T(1 \leq T \leq 100)$, the number of times you go on a pie-eating spree. For each case, there is first a line containing two space-separated integers, N and M $(1 \leq N, M \leq 300)$. Then, N lines follow, each containing M space-separated integers. The j-th integer on the ith line is $C_{i,j}(1 \leq C_{i,j} \leq 10^6)$.

Output

For the i-th case, print a line containing "`Case #i: `" followed by the minimum you need to pay to eat a pie every day.

Example

standard input	standard output
5 3 2 1 1 100 100 10000 10000 5 1 1 2 3 4 5 5 5 1 2 3 4 5 2 3 4 5 1 3 4 5 1 2 4 5 1 2 3 5 1 2 3 4 5 5 1 1 1 1 1 2 2 2 2 2 3 3 3 3 3 4 4 4 4 4 5 5 5 5 5 10 4 7 15 12 6 15 3 19 18 10 9 10 14 12 14 8 8 5 3 5 11 9 14 19 11 12 6 20 9 18 13 12 15 14 14 10 20 11 19 12 11	Case #1: 107 Case #2: 20 Case #3: 10 Case #4: 18 Case #5: 79

Note

In the first case, you should buy both pies on the first day, for a total cost of $1 + 1 + 2^2 = 6$. On the second day you should buy one pie for $100 + 1^2 = 101$. On the third day you can eat one of the spare pies you bought on the first day.

In the third case, you should buy and eat the cheapest pie every day, for a daily cost of $1 + 1^2 = 2$, and a total cost of 10.

In the fourth case, one possible solution is to buy two pies on the first day ($1 + 1 + 2^2 = 6$), two pies on the second day ($2 + 2 + 2^2 = 8$), and one pie on the third day ($3 + 1^2 = 4$) for a total cost of 18. On the fourth and fifth days you can eat your two spare pies from the first and second days.

B. Fighting the Zombies

Input file:	`standard input`
Output file:	`standard output`
Time limit:	15 seconds
Memory limit:	1024 megabytes

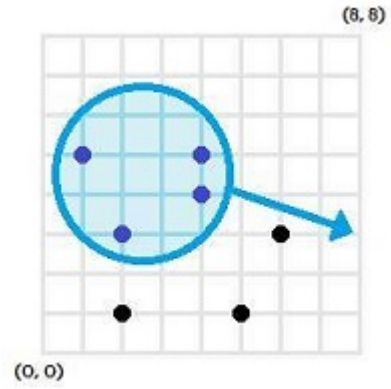
In the ever-popular role-playing game Dungeons & Dragons, or D&D, some wizards enhance their skills by training as geometers, specialist spell-casters who rely on the inherent magical properties of geometric forms.

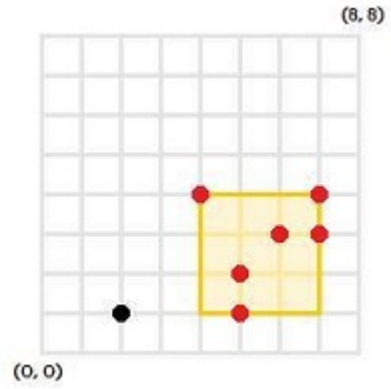
Unlike low-level wizards who may be able to fight only a single zombie at a time, you, the budding geometer, are ready and able to fight medium-sized hordes!

There are N zombies on an infinite 2D plane, with the ith one at coordinates (X_i, Y_i). You have two spells at your disposal, but you can only cast each one once.

You'll first cast the function-spell `Move(x, y, r, dx, dy)`. Each of its five parameters can be any real number, though r must be positive. Upon performing this spell, each zombie initially within the circle centered at coordinates (x, y) and having radius r will have its coordinates translated by (dx, dy). In other words, it will have dx added to its x-coordinate, and dy added to its y-coordinate. Zombies on the border of the circle are also affected.

You'll then cast the function-spell `Destroy(x, y)`. Each of its two parameters can be any real number. Upon performing this spell, each zombie currently within the axis-aligned square centered at coordinates (x, y) and having side-length R will be destroyed. Zombies on the border of the square are also affected.





What's the largest number of zombies that can be destroyed by this combination of function-spells?

Input

Input begins with an integer $T(1 \leq T \leq 50)$, the number of zombie hordes you fight. For each case, there is first a line containing two space-separated integers, $N(1 \leq N \leq 50)$ and $R(1 \leq R \leq 10^9)$. Then, N lines follow, the ith of which contains 2 space-separated integers, X_i and $Y_i(0 \leq X_i, Y_i \leq 10^9)$.

Output

For the ith case, print a line containing "`Case #i: `" followed by the maximum number of zombies you can slay with your spells.

It is guaranteed that no two zombies are initially at the same point, though zombies may later be moved such that there are multiple at the same coordinates.

Example

standard input	standard output
5 7 3 1 5 2 3 2 1 5 1 6 3 4 4 4 5 4 1 0 0 0 2 10 0 10 2 4 2 0 0 0 2 10 0 10 2 7 3 8 5 3 6 9 2 4 5 3 2 1 8 2 8 7 6 8 5 3 6 9 2 4 5 3 2 1 8 2 8	Case #1: 6 Case #2: 2 Case #3: 4 Case #4: 6 Case #5: 7

Note

In the first case (shown in the pictures above), one optimal strategy is to first cast `Move(2.5, 4.5, 2.2, 3, -1)`, moving 4 zombies, and then cast `Destroy(5.5, 2.5)`, destroying 6 zombies.

C. Manic Moving

Input file:	`standard input`
Output file:	`standard output`
Time limit:	15 seconds
Memory limit:	1024 megabytes

Thanks to his tireless hard work, Wilson has been promoted and now gets to drive his moving company's trucks! No, he can't believe it either.

The moving company services a region that has N towns, with M roads running amongst them. The i-th road connects two different towns A_i and B_i, requires G_i litres of gas to drive along, and can be traversed in either direction. There may be multiple roads running directly between any given pair of towns.

Today, Wilson has been scheduled to transport K families' belongings. The i-th family is moving from town S_i to a different town D_i. Wilson and his truck will be starting off the day at the company headquarters in town 1. For each family, he'll need to drive to their starting town by following a sequence of roads, load his truck there, and at some point later, arrive at their destination town to unload their belongings. His truck is large enough to fit at most 2 families' sets of belongings at a time, meaning that he doesn't necessarily need to deliver each load immediately after picking it up.

However, Wilson has been instructed that the K families must be helped strictly in order. In particular, if $i < j$, then the i-th family's belongings must be loaded before the j-th family's belongings are loaded, and the i-th family's belongings must be delivered before the j-th family's belongings are delivered.

Although Wilson's wages are higher than ever, he does have to pay for the truck's gas out of his own pocket, so it's in his best interest to get the job done while burning through as little of it as possible. Of course, he'll still need to be careful to follow his company's strict rules regarding the relative order of the families' loads and unloads, to avoid getting fired. That being said, it's a possibility for it to be impossible to even complete all of the requested moves, in which case Wilson will simply call it a day and stay home instead.

Input

Input begins with an integer $T(1 \leq T \leq 100)$, the number of sets of families Wilson needs to move.

For each case, there is first a line containing three space-separated integers, $N(2 \leq N \leq 100)$, $M(1 \leq M \leq 5000)$, and $K(1 \leq K \leq 5000)$.

Then, M lines follow, the ith of which contains 3 space-separated integers, A_i, B_i, and $G_i(1 \leq A_i, B_i \leq N, A_i \neq B_i, 1 \leq G_i \leq 1000)$.

Then, K lines follow, the ith of which contains 2 space-separated integers, S_i and $D_i(1 \leq S_i, D_i \leq N, S_i \neq D_i)$.

Output

For the ith case, print a line containing "`Case #i: `" followed by the minimum amount of gas required for Wilson to validly complete his delivery schedule, or "-1" if it can't be done.

Example

standard input	standard output
5 3 2 3 1 2 4 2 3 7 2 1 3 2 3 2 3 2 3 1 2 4 2 3 7 3 2 2 1 3 2 4 4 4 1 2 3 1 3 1 2 4 10 3 4 1 1 2 2 4 4 1 3 1 4 2 4 1 2 3 1 3 1 1 2 2 4 4 1 3 1 7 8 10 7 5 9 1 2 14 3 7 7 2 5 8 4 3 10 1 4 3 7 3 5 4 2 16 3 2 5 1 7 3 5 3 1 7 3 2 1 5 4 7 7 2 5 1	Case #1: 26 Case #2: 40 Case #3: 10 Case #4: -1 Case #5: 219

Note

In the first case, Wilson drives to town 2, and then drives the first family's belongings back to town 1. That's 8 litres gas so far. Then Wilson drives to city 3 (11 more litres of gas), picks up the remaining belongings, and drives them all to town 2 (7 litres of gas). A grand total of $8 + 11 + 7 = 26$ litres of gas.

In the fourth case, Wilson can't reach town 4 in order to complete the 2nd and 3rd families' moves.

D. Beach Umbrellas

Input file:	`standard input`
Output file:	`standard output`
Time limit:	15 seconds
Memory limit:	1024 megabytes

N groups of people are heading to the beach today! The i-th group is bringing a circular umbrella with a radius R_i meters.

The beach has M acceptable points at which umbrellas may be screwed into the sand, arranged in a line with 1 meter between each adjacent pair of points. Each group of people will choose one such point at which to position the center of their umbrella.

Of course, it's no good if any pair of umbrellas collide (that is, if the intersection of their circles has a positive area). The N groups will work together to place their umbrellas such that this doesn't happen. However, they're wondering in how many distinct ways that can be accomplished. Two arrangements are considered to be distinct if they involve at least one group placing their umbrella in a different spot. As this quantity may be very large, they're only interested in i-ts value modulo 1000000007 ($10^9 + 7$).

Note that it might be impossible for all of the groups to validly place their umbrellas, yielding an answer of 0.

Input

Input begins with an integer $T(1 \leq T \leq 100)$, the number of days the beach is open. For each day, there is first a line containing two space-separated integers, $N(1 \leq N \leq 2000)$ and $M(1 \leq M \leq 10^9)$. Then, N lines follow, the i-th of which contains a single integer, $R_i(1 \leq R_i \leq 2000)$.

Output

For the i-th day, print a line containing "`Case #i: `" followed by the number of valid umbrella arrangements, modulo 1000000007 ($10^9 + 7$).

Example

standard input	standard output
5 3 6 1 1 1 2 5 1 2 3 2000 1 2 3 5 22 1 2 3 4 5 1 10 50	Case #1: 24 Case #2: 6 Case #3: 916295601 Case #4: 12 Case #5: 10

Note

In the second case there are six possibilities. If the radius-1 umbrella is placed at the far-left point, then the radius-2 umbrella can be placed at either of the two right-most points. If the radius-1 umbrella is placed at the second point from the left, then the radius-2 umbrella must be placed at the right-most point. That's three possibilities so far, and we can mirror them to produce three more.